AdaProp: Learning Adaptive Propagation for Graph Neural Network based Knowledge Graph Reasoning

Yongqi Zhang*
zhangyongqi@4paradigm.com
4Paradigm Inc.
Beijing, China

Zhanke Zhou*
cszkzhou@comp.hkbu.edu.hk
Department of Computer Science,
Hong Kong Baptist University
Hong Kong, China

Quanming Yao†
qyaoaa@tsinghua.edu.cn
Department of Electronic
Engineering, Tsinghua University
Beijing, China

Xiaowen Chu
xwchu@ust.hk
Data Science and Analytics Thrust,
HKUST (Guangzhou)
Guangzhou, China

Bo Han
bhanml@comp.hkbu.edu.hk
Department of Computer Science,
Hong Kong Baptist University
Hong Kong, China

ABSTRACT

Due to the popularity of Graph Neural Networks (GNNs), various GNN-based methods have been designed to reason on knowledge graphs (KGs). An important design component of GNN-based KG reasoning methods is called the propagation path, which contains a set of involved entities in each propagation step. Existing methods use hand-designed propagation paths, ignoring the correlation between the entities and the query relation. In addition, the number of involved entities will explosively grow at larger propagation steps. In this work, we are motivated to learn an adaptive propagation path in order to filter out irrelevant entities while preserving promising targets. First, we design an incremental sampling mechanism where the nearby targets and layer-wise connections can be preserved with linear complexity. Second, we design a learning-based sampling distribution to identify the semantically related entities. Extensive experiments show that our method is powerful, efficient and semantic-aware. The code is available at https://github.com/LARS-research/AdaProp.

CCS CONCEPTS

• **Computing methodologies** → **Knowledge representation and reasoning**.

KEYWORDS

Knowledge graph, Graph embedding, Knowledge graph reasoning, Graph sampling, Graph neural network

*This work was partially performed when Z. Zhou was an intern in 4Paradigm.
Y. Zhang and Z. Zhou have equal contribution, and correspondence is to Q. Yao.

KDD '23, August 6–10, 2023, Long Beach, CA, USA

ACM ISBN 979-8-4007-0103-0/23/08…$15.00
https://doi.org/10.1145/3580305.3599404

ACM Reference Format:
Yongqi Zhang, Zhanke Zhou, Quanming Yao, Xiaowen Chu, and Bo Han. 2023. AdaProp: Learning Adaptive Propagation for Graph Neural Network based Knowledge Graph Reasoning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23), August 6–10, 2023, Long Beach, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3580305.3599404

1 INTRODUCTION

Knowledge graph (KG) is a semantic graph representing the relationships between real-world entities [16, 39]. Reasoning on KGs aims to deduce missing answers for given queries based on existing facts [1, 6, 20]. It has been widely applied in drug interaction prediction [49], personalized recommendation [4, 40], and question answering [12, 18]. An example of KG reasoning with query *(LeBron, lives_in, ?)* and answer *Los Angeles* is illustrated in Figure 1. Symbolically, the reasoning problem can be denoted as a *query* $(e_q, r_q, ?)$, with the query entity e_q and query relation r_q. The objective of KG reasoning is to find the target answer entity e_a for $(e_q, r_q, ?)$ by the local evidence learned from the given KG.

Representative methods for KG reasoning can be classified into three categories: (i) triplet-based models [3, 10, 39, 52, 55] directly score each answer entity through the learned entity and relation embeddings; (ii) path-based methods [7, 9, 30, 31, 54] learn logic rules to generate the relational paths starting from e_q and explore which entity is more likely to be the target answer e_a; (iii) GNN-based methods [32, 38, 53, 56] propagate entity representations among the local neigborhoods by graph neural networks [22]. Among the three categories, the GNN-based methods achieve the state-of-the-art performance [47, 53, 56]. Specifically, R-GCN [32] and CompGCN [38], propagate in the full neighborhoods over all the entities. Same as the general GCNs, they will easily suffer from over-smoothing problem [28]. NBFNet [56] and RED-GNN [53] propagates in the local neighborhoods around the query entity e_q. By selecting the range of propagation, they possess the stronger reasoning ability than R-GCN or CompGCN. However, their local neighborhoods mainly depend on the query entity, and the majority of entities will be involved at larger propagation steps.

The existing GNN-based methods, which propagate among all the neighborhood over all the entities or around the query entity,

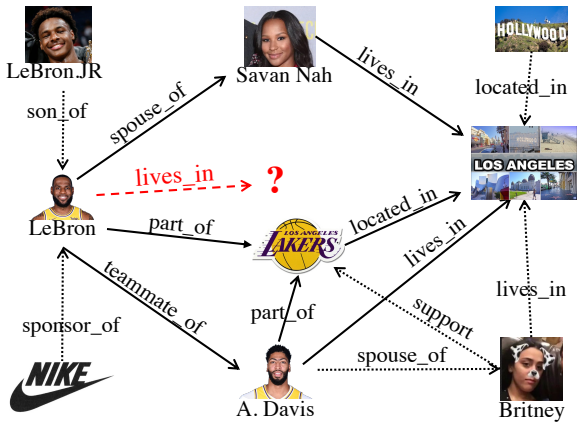


Figure 1: An example of KG reasoning. The query *(Lebron, lives_in,?)* asks which city *Lebron* lives in. Dashed lines are irrelevant facts to this query while solid lines are relevant.

ignore the semantic relevance between local neighborhoods and the query relation. Intuitively, the local information we need depends on both the query entity and query relation. For example, the most relevant local evidence to the query in Figure 1 includes the combination of *(LeBron, part_of, Lakers)* & *(Lakers, located_in, L.A.)* and *(LeBron, spouse_of, Savan Nah)* & *(Savan Nah, lives_in, L.A.)*. If the query changes to *(LeBron, father_of, ?)*, the most relevant information will be changed to *(LeBron JR, son_of, LeBron)*. In addition, the existing GNN-based methods will inevitably involve too many irrelevant entities and facts, increasing the learning difficulty and computing cost, especially on large-scale KGs.

Motivated by above examples, we propose to adaptively sample semantically relevant entities during propagation. There are three challenges when designing the sampling algorithm: (i) the structure of KG is complex and the size is large; (ii) the KG is multi-relational where the edges and queries have different kinds of relations, representing different semantic meanings; (iii) there is no direct supervision and simple heuristic measurement to indicate the semantic relevance of entities to the given query. To address these challenges, we propose AdaProp, a GNN-based method with an adaptive propagation path. The key idea is to reduce the number of involved entities while preserving the relevant ones in the propagation path. This is achieved by an incremental sampling mechanism and a learning-based sampling distribution that adapts to different queries. In this way, our method learn an adaptive propagation path that preserves layer-wise connection, selects semantic related entities, with fewer reduction in the promising targets. The main contributions are summarized as follows:

- We propose a connection-preserving sampling scheme, called incremental sampling, which only has linear complexity with regard to the propagation steps and can preserve the layer-wise connections between sampled entities.
- We design a semantic-aware Gumble top-k distribution which adaptively selects local neighborhoods relevant to the query relation and is learned by a straight through estimator.
- Experiments show that the proposed method achieves the state-of-the-art performance in both transductive and inductive KG reasoning settings. It can preserve more target entities than the other samplers. And the case study shows that the learned sampler is query-dependent and semantic-aware.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{R}, \mathcal{E}, \mathcal{Q})$ be an instance of KG, where $\mathcal{V}$, $\mathcal{R}$ are the sets of entities and relations, $\mathcal{E} = \{(e_s, r, e_o) | e_s, e_o \in \mathcal{V}, r \in \mathcal{R}\}$ is the set of fact triplets, and $\mathcal{Q} = \{(e_q, r_q, e_a) | e_q, e_a \in \mathcal{V}, r \in \mathcal{R}\}$ is the set of query triplets with [1] query $(e_q, r_q, ?)$ and target answer entity e_a. The reasoning task is to learn a function that predicts the answer $e_a \in \mathcal{V}$ for each query $(e_q, r_q, ?)$ based on the fact triplets $\mathcal{E}$.

2 RELATED WORKS

2.1 GNN for KG reasoning

Due to the success of graph neural networks (GNNs) [14, 22] in modeling the graph-structured data, recent works [36, 38, 53, 56] attempt to utilize the power of GNNs for KG reasoning. They generally follow the message propagation framework [14, 22] to propagate messages among entities. In the l-th propagation step, specifically, the messages are firstly computed on edges $\mathcal{E}^\ell = \{(e_s, r, e_o) \in \mathcal{E} | e_s \in \mathcal{V}^{\ell-1}), r \in \mathcal{R}, e_o \in \mathcal{V}^\ell\}$ with message function $\boldsymbol{m}^\ell_{(e_s,r,e_o)} := \text{MESS}\left(\boldsymbol{h}^{\ell-1}_{e_s}, \boldsymbol{h}^\ell_r\right)$ based on entity representation $\boldsymbol{h}^{\ell-1}_{e_s}$ and relation representation $\boldsymbol{h}^\ell_r$. And then the messages are propagated from entities of previous step ($e_s \in \mathcal{V}^{\ell-1}$) to entities of current step ($e_o \in \mathcal{V}^\ell$) via $\boldsymbol{h}^\ell_{e_o} := \delta\left(\text{AGG}(\boldsymbol{m}^\ell_{(e_s,r,e_o)}, (e_s, r, e_o) \in \mathcal{E}^\ell)\right)$, with an activation function $\delta(\cdot)$. The specified functions MESS$(\cdot)$ and AGG$(\cdot)$ for different methods are provided in Appendix A.1. After L steps' propagation, the entity representations $\boldsymbol{h}^L_{e_o}$ are obtained to measure the plausibility of each entity $e_o \in \mathcal{V}^L$.

For the GNN-based KG reasoning methods, the sets of entities participated in propagation are different. The existing GNN-based methods can be generally classified into three classes based on their design of propagation range:

- *Full* propagation methods, e.g., R-GCN [32] and CompGCN [38], propagate among all the entities, i.e., $\mathcal{V}^\ell = \mathcal{V}$. They are limited to small propagation steps due to the large memory cost and the over-smoothing problem of GNNs [28] over full neighbors.
- *Progressive* propagation methods (RED-GNN [53] and NBFNet [56]), propagate from the query entity e_q and progressively to the L-hop neighborhood of e_q, i.e., $\mathcal{V}^0 = \{e_q\}$ and $\mathcal{V}^\ell = \bigcup_{e \in \mathcal{V}^{\ell-1}} \mathcal{N}(e)$, where $\mathcal{N}(e)$ contains the 1-hop neighbors of entity e.
- *Constrained* propagation methods, e.g., GraIL [36] and CoMPILE [25], propagate within a constrained range, i.e., $\mathcal{V}^\ell = \mathcal{V}^L_{e_q,e_a}$, where $\mathcal{V}^L_{e_q,e_a} \subset \mathcal{V}$ is the enclosing subgraph for e_q and e_a. Since $\mathcal{V}^L_{e_q,e_a}$ are different for different pairs of (e_q, e_a), these methods are extremely expensive, especially on large-scale KGs (see discussion in Appendix A.2).

We provide graphical illustration of propagation scheme for the three categories in the middle of Figure 2. Among them, the progressive propagation methods achieved state-of-the-art reasoning performance. They are much more efficient than the constrained propagation methods, and the entities used for propagation depend on the query entity, filtering out many irrelevant entities. However, $\mathcal{V}^L$ of progressive methods will also involve in a majority or even all of the entities when L gets large.

[1] Note that predicting missing head in KG can also be formulated in this way by adding inverse relations (details in Appendix B.1).

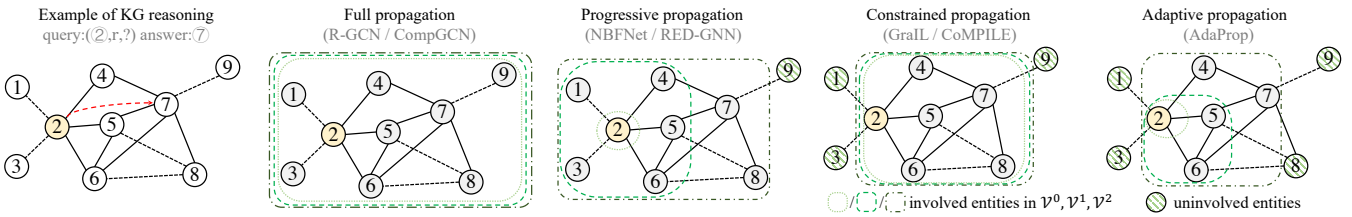


Figure 2: Symbolic illustrations of the KG in Figure 1 and the different designs of propagation path. For simplicity, we use circles to represent entities and ignore the types of relation. Specifically, (1) full propagation propagates over the full set of entities $\mathcal{V}$; (2) progressive propagation starts from the query entity e_q and gradually propagates to its ℓ-hop neighbors in the ℓ-th step; (3) constrained propagation propagates within a constrained range $\mathcal{V}^L_{e_q,e_a}$ with entities less than 2 distance away from both e_q and e_a; (4) adaptive propagation of AdaProp starts from e_q and adaptively selects the semantic-relevant entities.

2.2 Sampling methods for GNN

The sampling methods can control and select entities during propagation. There are several methods introduced to sample nodes on homogeneous graphs with the objective to improve the scalability of GNNs. They can be classified into three categories: (i) node-wise sampling methods, such as GraphSAGE [15] and PASS [48], sample K entities from the neighbor set $\mathcal{N}(e)$ for each entity e in each propagation step; (ii) layer-wise sampling methods, like FastGCN [5], Adaptive-GCN [17] and LADIES [57], sample at most K entities in each propagation step; (iii) subgraph sampling methods, such as Cluster-GCN [8], GraphSAINT [51] and ShadowGNN [50], directly extract the local subgraph around the queried entity. All these methods are designed for homogeneous graphs with the purpose of solving scalability issues.

There are also a few sampling methods proposed for KG reasoning. RS-GCN [13] proposes a node-wise sampling method to select neighbors for R-GCN/CompGCN. DPMPN [45] contains a full propagation GNN to learn the global feature, and prunes another GNN for reasoning in a layer-wise sampling manner. These sampling approaches reduce the memory cost for propagation on larger KGs, but their empirical performance does not gain much compared with the base models.

3 THE PROPOSED METHOD

3.1 Problem formulation

We denote a GNN model for KG reasoning as $F(\boldsymbol{w}, \widehat{\mathcal{G}}^L)$ with model parameters $\boldsymbol{w}$. $\widehat{\mathcal{G}}^L$ here is called the *propagation path*, containing the sets of involved entities in each propagation step, i.e., $\widehat{\mathcal{G}}^L = \{\mathcal{V}^0, \mathcal{V}^1, ..., \mathcal{V}^L\}$ where $\mathcal{V}^\ell, \ell = 1...L$ is the set of involved entities in the ℓ-th propagation step. The representations of entities are propagated from $\mathcal{V}^0$ to $\mathcal{V}^L$ in $\widehat{\mathcal{G}}^L$ for multiple steps.

The objective is to design better propagation path $\widehat{\mathcal{G}}^L$ such that the model with optimized parameters can correctly predict target answer entity e_a for each query $(e_q, r_q, ?)$. The progressive propagation methods have different propagation path according to the query entities, and achieved state-of-the-art performance. However, they ignore the dependency of query relations and will involve in too many entities at larger propagation steps. Motivated by the success of progressive propagation methods and sampling methods on homogeneous graph, we aim to use sampling technique to dynamically adapt the propagation paths. Formally, we define the *query-dependent propagation path* for a query $(e_q, r_q, ?)$ as

$$\widehat{\mathcal{G}}^L_{e_q,r_q} = \{\mathcal{V}^0_{e_q,r_q}, \mathcal{V}^1_{e_q,r_q}, ..., \mathcal{V}^L_{e_q,r_q}\},$$
$$\text{s.t. } \mathcal{V}^\ell_{e_q,r_q} = \begin{cases} \{e_q\} & \ell = 0 \\ S(\mathcal{V}^{\ell-1}_{e_q,r_q}) & \ell = 1...L \end{cases}. \quad (1)$$

The propagation here starts from the query entity e_q, and the entities in $\mathcal{V}^\ell_{e_q,r_q}$ is adaptively sampled as a subset of $\mathcal{V}^{\ell-1}_{e_q,r_q}$ and its neighbors with a sampling strategy $S(\cdot)$.

The key problem here is how to design the sampling strategy $S(\cdot)$. For the KG reasoning task, we face two main challenges:

- the target answer entity e_a is unknown given the query $(e_q, r_q, ?)$, thus directly sampling neighboring entities may loss the connection between query entity and target answer entity;
- the semantic dependency between entities in the propagation path and the query relation r_q is too complex to be captured by simple heuristics.

The existing sampling methods are not very applicable since (i) they do not consider the preserving of unknown target entities; (ii) they are designed for homogeneous graph without modeling relation types; and (iii) there is no direct supervision on the dependency of entities in the propagation on the query. Therefore, we would argue that a specially designed sampling approach is needed for KG reasoning, which is expected to remove entities and also preserving the promising targets for corresponding query.

In the following parts, we introduce the adaptive propagation (AdaProp) algorithm, which can adaptively select relevant entities into the propagation path based on the given query. To solve the first challenge, we design a connection-preserving sampling scheme, called incremental sampling, which can reduce the number of involved entities and preserve the layer-wise connections in Section 3.2. For the second challenge, we propose a relation-dependent sampling distribution, which is jointly optimized with the model parameters, to select entities that are semantically relevant to the query relation in Section 3.3. The full algorithm is summarized in Section 3.4.

3.2 The connection-preserving incremental sampling strategy

When designing the sampling strategy, we should consider what to be sampled, and how to sample effectively and efficiently. Since the target entity e_a is unknown given the query $(e_q, r_q, ?)$, freely sampling from all the possible neighbors will loss the structural

connection between query entity and promising targets. The existing sampling methods cannot well address this problem. For the node-wise sampling methods, the complexity grows exponentially w.r.t. propagation steps L. For the layer-wise sampling methods, the entities in previous step may have little connection with the sampled entities in the current step, failing to preserve the local structures. The subgraph sampling methods can better preserve local structure, but is harder to control which entities are more relevant to the query and deserve to be sampled.

Observing that the majority of the target answer entities lie close to the query entity (see the distributions in Appendix C.1), we are motivated to preserve the entities already been selected in each propagation step. In other words, we use the constraint $\mathcal{V}^0_{e_q,r_q} \subseteq \mathcal{V}^1_{e_q,r_q} \cdots \subseteq \mathcal{V}^L_{e_q,r_q}$ to preserve the previous step entities and sample from the newly-visited ones in an incremental manner to avoid dropping the promising targets. On one hand, the nearby targets will be more likely to be preserved when the propagation steps get deeper. On the other hand, the layer-wise connection between entities in two consecutive layers can be preserved. The incremental sampler $S(\cdot) = \texttt{SAMP}(\texttt{CAND}(\cdot))$ contains two parts, i.e., candidate generation $\texttt{CAND}(\cdot)$ and candidate sampling $\texttt{SAMP}(\cdot)$.

3.2.1 Candidate generation. Given the set of entities in the $(\ell-1)$-th step $\mathcal{V}^{\ell-1}_{e_q,r_q}$, we denote the candidate entities to sample in the ℓ-th step as $\overline{\mathcal{V}}^{\ell}_{e_q,r_q} := \texttt{CAND}(\mathcal{V}^{\ell-1}_{e_q,r_q})$. Basically, all the neighboring entities in $\mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q}) = \bigcup_{e\in\mathcal{V}^{\ell-1}_{e_q,r_q}} \mathcal{N}(e)$ can be regarded as candidates for sampling. However, to guarantee that $\mathcal{V}^{\ell-1}_{e_q,r_q}$ will be preserved in the next sampling step, we regard the newly-visited entities as the candidates, i.e.,

$$\overline{\mathcal{V}}^{\ell}_{e_q,r_q} := \texttt{CAND}(\mathcal{V}^{\ell-1}_{e_q,r_q}) = \mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q}) \setminus \mathcal{V}^{\ell-1}_{e_q,r_q}.$$

For example, given the entity ② in Figure 2, we generate the 1-hop neighbors ①, ③, ④, ⑤, ⑥ as candidates and directly preserve ② in the next propagation step.

3.2.2 Candidate sampling. To control the number of involved entities, we sample K ($\ll |\overline{\mathcal{V}}^{\ell}_{e_q,r_q}|$) entities without replacement from the candidate set $\overline{\mathcal{V}}^{\ell}_{e_q,r_q}$. The sampled entities at ℓ-th step together with entities in the $(\ell-1)$-th step constitute the involved entities at the ℓ step, i.e.,

$$\mathcal{V}^{\ell}_{e_q,r_q} := \mathcal{V}^{\ell-1}_{e_q,r_q} \cup \texttt{SAMP}(\overline{\mathcal{V}}^{\ell}_{e_q,r_q}).$$

Given the candidates of entity ② in Figure 2, the entities ⑤ and ⑥ are sampled and activated from the five candidates in $\overline{\mathcal{V}}^{\ell}_{e_q,r_q}$. Along with the previous layer entity ②, the three entities are collected as $\mathcal{V}^{\ell}_{e_q,r_q}$ for next step propagation.

3.2.3 Discussion. There are three advantages of the incremental sampling strategy. First, it is more entity-efficient than the node-wise sampling methods, which have exponentially growing number of entities over propagation steps. As in Prop.1, the number of involved entities grows linearly w.r.t. the propagation steps L. Second, the layer-wise connection can be preserved based on Prop.2, which shows advantage over the layer-wise sampling methods. Third, as will be shown in Section 4.2.1, it has larger probability to preserve the target answer entities than the other sampling mechanisms with the same amount of involved entities.

Proposition 1. *The number of involved entities in the propagation path ($|\bigcup_{\ell=0\dots L}\mathcal{V}^{\ell}_{e_q,r_q}|$) of incremental sampling is bounded by $O(LK)$.*

Proposition 2. *For all the entities $e \in \mathcal{V}^{\ell-1}_{e_q,r_q}$ with incremental sampling, there exists at least one entity $e' \in \mathcal{V}^{\ell}_{e_q,r_q}$ and relation $r \in \mathcal{R}$ such that $(e, r, e') \in \mathcal{E}$.*

3.3 Learning semantic-aware distribution

The incremental sampling mechanism alleviates the problem of reducing target candidates. However, randomly sampling K entities from $\overline{\mathcal{V}}^{\ell}_{e_q,r_q}$ does not consider the semantic relevance of different entities for different queries. The greedy-based search algorithms like beam search [42] are not applicable here as there is no direct heuristic to measure the semantic relevance. To solve this problem, we parameterize the sampler $S(\cdot)$ with parameter $\boldsymbol{\theta}^{\ell}$ for each propagation step ℓ, i.e., $\mathcal{V}^{\ell}_{e_q,r_q} = S(\mathcal{V}^{\ell-1}_{e_q,r_q}; \boldsymbol{\theta}^{\ell})$. In the following two parts, we introduce the sampling distribution and how to optimize the sampling parameters.

3.3.1 Parameterized sampling distribution. For the GNN-based KG reasoning methods, the entity representations $\boldsymbol{h}^L_{e_o}$ in the final propagation step are used to measure the plausibility of entity e_o being the target answer entity. In other words, the entity representations are learned to indicate their relevance to the given query $(e_q, r_q, ?)$. To share this knowledge, we introduce a linear mapping function $g(\boldsymbol{h}^{\ell}_{e_o}; \boldsymbol{\theta}^{\ell}) = \boldsymbol{\theta}^{\ell\top}\boldsymbol{h}^{\ell}_{e_o}$ with parameters $\boldsymbol{\theta}^{\ell} \in \mathbb{R}^d$ in each layer $\ell = 1 \dots L$, and sample according to the probability distribution

$$p^{\ell}(e) := \exp\big(g(\boldsymbol{h}^{\ell}_{e}; \boldsymbol{\theta}^{\ell})/\tau\big) \Big/ \sum\nolimits_{e'\in\overline{\mathcal{V}}^{\ell}_{e_q,r_q}} \exp\big(g(\boldsymbol{h}^{\ell}_{e'}; \boldsymbol{\theta}^{\ell})/\tau\big), \quad (2)$$

with temperature value $\tau > 0$.

Sampling K entities without replacement from $p^{\ell}(e)$ requires to sequentially sample and update the distribution for each sample, which can be expensive. We follow the Gumbel top-k trick [24, 43] to solve this issue. The Gumbel-trick firstly samples $|\overline{\mathcal{V}}^{\ell}_{e_q,r_q}|$ independent noises from the uniform distribution, i.e., $U_e \sim \text{Uniform}(0, 1)$ to form the Gumbel logits $G_e = g(\boldsymbol{h}^{\ell}_{e}; \boldsymbol{\theta}^{\ell}) - \log(-\log U_e)$ for each $e \in \overline{\mathcal{V}}^{\ell}_{e_q,r_q}$. The top-$K$ entities in $\overline{\mathcal{V}}^{\ell}_{e_q,r_q}$ are then collected based on their values of Gumbel logits G_e. As proved in [24, 43], this procedure is equivalent to sample without replacement from $p^{\ell}(e)$.

3.3.2 Learning strategy. We denote $\boldsymbol{\theta} = \{\boldsymbol{\theta}^1, \dots, \boldsymbol{\theta}^L\}$ as the sampling parameters and the parameterized propagation path as $\widehat{\mathcal{G}}^L_{e_q,r_q}(\boldsymbol{\theta})$. Since our sampling distribution in (2) is strongly correlated with the GNN representations, jointly optimizing both the model parameters and sampler parameters can better share knowledge between the two parts. Specifically, we design the following objective

$$\boldsymbol{w}^*, \boldsymbol{\theta}^* = \arg\min_{\boldsymbol{w},\boldsymbol{\theta}} \sum\nolimits_{(e_q,r_q,e_a)\in\mathcal{Q}_{\text{tra}}} \mathcal{L}\big(F(\boldsymbol{w}, \widehat{\mathcal{G}}^L_{e_q,r_q}(\boldsymbol{\theta})), e_a\big). \quad (3)$$

The loss function $\mathcal{L}$ on each instance is a binary cross entropy loss on all the entities $e_o \in \mathcal{V}^L_{e_q,r_a}$, i.e.,

$$\mathcal{L}\big(F(\boldsymbol{w}, \widehat{\mathcal{G}}^L(\boldsymbol{\theta})), e_a\big) = -\sum\nolimits_{e_o\in\mathcal{V}^L_{e_q,r_a}} y_{e_o}\log(\phi_{e_o}) + (1-y_{e_o})\log(1-\phi_{e_o}),$$

Algorithm 1 AdaProp: learning adaptive propagation path.

Require: query $(e_q, r_q, ?)$, $\mathcal{V}^0_{e_q,r_q} = \{e_q\}$, steps L, number of sampled entities K, and functions $\mathrm{MESS}(\cdot)$ and $\mathrm{AGG}(\cdot)$.

1: **for** $\ell = 1 \dots L$ **do**
2: get the neighboring entities $\mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q}) = \cup_{e\in\mathcal{V}^{\ell-1}_{e_q,r_q}} \mathcal{N}(e)$, the newly-visited entities $\overline{\mathcal{V}}^{\ell}_{e_q,r_q} = \mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q}) \setminus \mathcal{V}^{\ell-1}_{e_q,r_q}$, and edges $\mathcal{E}^{\ell} = \{(e_s, r, e_o) | e_s \in \mathcal{V}^{\ell-1}_{e_q,r_q}, e_o \in \mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q})\}$;
3: obtain $\boldsymbol{m}^{\ell}_{(e_s,r,e_o)} := \mathrm{MESS}(\boldsymbol{h}^{\ell-1}_{e_s}, \boldsymbol{h}^{\ell-1}_{e_o}, \boldsymbol{h}^{\ell}_{r}, \boldsymbol{h}^{\ell}_{r_q})$ for edges $(e_s, r, e_o) \in \mathcal{E}^{\ell}$;
4: obtain $\boldsymbol{h}^{\ell}_{e_o} := \delta\big(\mathrm{AGG}\big(\boldsymbol{m}^{\ell}_{(e_s,r,e_o)}, (e_s, r, e_o) \in \mathcal{E}^{\ell}\big)\big)$ for entities $e_o \in \mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q})$;
5: **logits computation:** obtain the Gumbel logits $G_{e_o} = g(\boldsymbol{h}^{\ell}_{e_o}; \boldsymbol{\theta}^{\ell}) - \log(-\log U_{e_o})$ with $U_{e_o} \sim \mathrm{Uniform}(0, 1)$ for entities $e_o \in \overline{\mathcal{V}}^{\ell}_{e_q,r_q}$;
6: **candidate sampling:** obtain sampled entities $\widetilde{\mathcal{V}}^{\ell}_{e_q,r_q} = \{\arg\mathrm{top}_K G_{e_o}, e_o \in \overline{\mathcal{V}}^{\ell}_{e_q,r_q}\}$;
7: **straight-through:** $\boldsymbol{h}^{\ell}_{e} := \big(1 - \mathtt{no\_grad}(p^{\ell}(e)) + p^{\ell}(e)\big) \cdot \boldsymbol{h}^{\ell}_{e}$ for entities $e \in \widetilde{\mathcal{V}}^{\ell}_{e_q,r_q}$;
8: **update propagation path:** update $\mathcal{V}^{\ell}_{e_q,r_q} = \mathcal{V}^{\ell-1}_{e_q,r_q} \cup \widetilde{\mathcal{V}}^{\ell}_{e_q,r_q}$;
9: **end for**
10: **return** $f(\boldsymbol{h}^{L}_{e_o}; \boldsymbol{w}^{\top})$ for each $e_o \in \mathcal{V}^{L}_{e_q,r_q}$.

where the likelihood score $\phi_{e_o} = f(\boldsymbol{h}^{L}_{e_o}; \boldsymbol{w}^{\top}_{\phi}) \in [0, 1]$ for entity e_o indicating the plausibility of e_o being the target answer entity, and the label $y_{e_o} = 1$ if $e_o = e_a$ otherwise 0.

The reparameterization trick is often used for Gumbel distributions [19]. However, it does not do explicit sampling, thus still requires high computing cost. Instead, we choose the straight-through (ST) estimator [2, 19], which can approximately estimate gradient for discrete variables. The key idea of ST-estimator is to back-propagate through the sampling signal as if it was the identity function. Specifically, rather than directly use the entity representations $\boldsymbol{h}^{\ell}_{e}$ to compute messages, we use $\boldsymbol{h}^{\ell}_{e} := \big(1 - \mathtt{no\_grad}(p^{\ell}(e)) + p^{\ell}(e)\big) \cdot \boldsymbol{h}^{\ell}_{e}$, where $\mathtt{no\_grad}(p^{\ell}(e))$ means that the back-propagation signals will not go through this term. In this approach, the forward computation will not be influenced, while the backward signals can go through the probability multiplier $p^{\ell}(e)$. An alternative choice to estimate the gradient on discrete sampling distribution is the REINFORCE technique [27, 41]. However, REINFORCE gradient is known to have high variance [35] and the variance will be accumulated across the propagation steps, thus not used here. The model parameters $\boldsymbol{w}$ and sampler parameters $\boldsymbol{\theta}$ are simultaneously updated by Adam optimizer [21].

Overall, the objective of learning semantic-aware distribution is to adaptively preserve the promising targets according to the query relation. As will be shown in Section 4.3. the selected information in the sampled propagation path are semantically related to the query relation by sharing knowledge with the entity representations. On the other hand, the learned distribution guarantees a higher coverage of target entities compared with the not learned version.

3.4 The full algorithm

The full procedure of AdaProp is shown in Algorithm 1. Given the entities $\mathcal{V}^{\ell-1}_{e_q,r_q}$ in the $(\ell-1)$-th step, we first obtain the neighboring entities $\mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q})$ of $\mathcal{V}^{\ell-1}_{e_q,r_q}$, and generate the newly-visited ones $\overline{\mathcal{V}}^{\ell}_{e_q,r_q} = \mathcal{N}(\mathcal{V}^{\ell-1}) \setminus \mathcal{V}^{\ell-1}_{e_q,r_q}$ as candidates in line 2. Then, query-dependent messages are computed on edges $\mathcal{E}^{\ell}$ in line 3, and propagated to entities in $\mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q})$ in line 4. The top-K entities in $\overline{\mathcal{V}}^{\ell}_{e_q,r_q}$ are sampled in line 6 according to the Gumbel logits computed in line 5. As for the ST-estimator, we modify the hidden representations for entities in $\widetilde{\mathcal{V}}^{\ell}_{e_q,r_q}$ in line 7. Finally, the set of sampled entities $\widetilde{\mathcal{V}}^{\ell}_{e_q,r_q}$ are concatenated with $\mathcal{V}^{\ell-1}_{e_q,r_q}$ to form the set of ℓ-th step entities $\mathcal{V}^{\ell}_{e_q,r_q}$ in line 8. After propagating for L steps, the final step representations $\boldsymbol{h}^{L}_{e_o}$ for all the involved entities are returned. The entity with the highest score evaluated by $f(\boldsymbol{h}^{L}_{e_o}; \boldsymbol{w}_{\phi})$ in $\mathcal{V}^{L}_{e_q,r_q}$ is regarded as the predicted target.

The key difference compared with existing GNN-based KG reasoning methods is that the propagation path in Algorithm 1 is no longer given and fixed before the message propagation. Instead, it is adaptively adjusted based on the entity representations and is dynamically updated in each propagation step. In addition, Algorithm 1 also enjoys higher efficiency than the full propagation and progressive propagation methods, since only a small portion of entities are involved, while the extra computation cost for sampling is relatively low. Compared with constrained propagation methods, which needs to propagate in the answer-specific propagation paths for multiple times, the potential target answer entities are directly scored in the final propagation step in a single forward pass.

4 EXPERIMENTS

In this section, we empirically verify the effectiveness and analyze the components of the proposed AdaProp on both transductive and inductive settings. All the experiments are implemented in Python with PyTorch [29] and run on a single NVIDIA RTX 3090 GPU with 24GB memory.

4.1 Comparison with KG reasoning methods

We compare AdaProp with general KG reasoning methods in both transductive and inductive reasoning. In the transductive setting, the entities sets are the same for training and testing. While in the testing of inductive reasoning, the model are required to generalize to unseen entities w.r.t. the training set. Besides, we follow [3, 36, 39, 53, 56] to use the filtered ranking-based metrics for evaluation, i.e., mean reciprocal ranking (MRR) and Hit@k (e.g., Hit@1 and Hit@10). The higher value indicates the better performance for both metrics. As for the hyper-parameters, we tune the number of propagation steps L from 5 to 8, the number of sampled entities K in $\{100, 200, 500, 1000, 2000\}$, and the temperature value τ in $\{0.5, 1.0, 2.0\}$. The ranges of other hyper-parameters are kept the same as RED-GNN [53].

4.1.1 *Transductive setting.*

Figure 1 shows an example of transductive KG reasoning, where a KG related to *Lakers* is given and the task is to predict where *LeBron* lives in based on the given facts. Formally speaking, we have one KG $\mathcal{G} = (\mathcal{V}, \mathcal{R}, \mathcal{E}, \mathcal{Q})$ where the query set $\mathcal{Q}$ is split into three disjoint sets $\mathcal{Q}_{\mathrm{tra}}/\mathcal{Q}_{\mathrm{val}}/\mathcal{Q}_{\mathrm{tst}}$ for training, validation and testing respectively.

Datasets. We use six widely used KG completion datasets, including Family [23], UMLS [23], WN18RR [10], FB15k237 [37], NELL-995 [44] and YAGO3-10 [33]. The statistics of these datasets are provided in Table 1.

Table 1: Statistics of the transductive KG datasets. Fact triplets in $\mathcal{E}$ are used to build the graph, and $\mathcal{Q}_{\text{tra}}$, $\mathcal{Q}_{\text{val}}$, $\mathcal{Q}_{\text{tst}}$ are the query triplets used for reasoning.

dataset	#entity	#relation	$\lvert\mathcal{E}\rvert$	$\lvert\mathcal{Q}_{\text{tra}}\rvert$	$\lvert\mathcal{Q}_{\text{val}}\rvert$	$\lvert\mathcal{Q}_{\text{tst}}\rvert$
Family	3.0k	12	23.4k	5.9k	2.0k	2,8k
UMLS	135	46	5.3k	1.3k	569	633
WN18RR	40.9k	11	65.1k	21.7k	3.0k	3.1k
FB15k237	14.5k	237	204.1k	68.0k	17.5k	20.4k
NELL-995	74.5k	200	112.2k	37.4k	543	2.8k
YAGO3-10	123.1k	37	809.2k	269.7k	5.0k	5.0k

Baselines. We compare the proposed AdaProp with (i) non-GNN methods ConvE [10], QuatE [52], RotatE [34], MINERVA [9], DRUM [31], RNNLogic [30] and RLogic [7]; and (ii) GNN-based full propagation method CompGCN [38] and progressive propagation methods NBFNet [56] and RED-GNN [53] here. R-GCN [32] is not compared here as it is much inferior to CompGCN [38]. As indicated by [53, 56], GraIL [36] and CoMPILE [25] are intractable to work on the transductive KGs with many entities, and thus not compared in this setting. Results of these baselines are taken from their papers or reproduced by their official codes. Missing results of RLogic are due to the lack of source code.

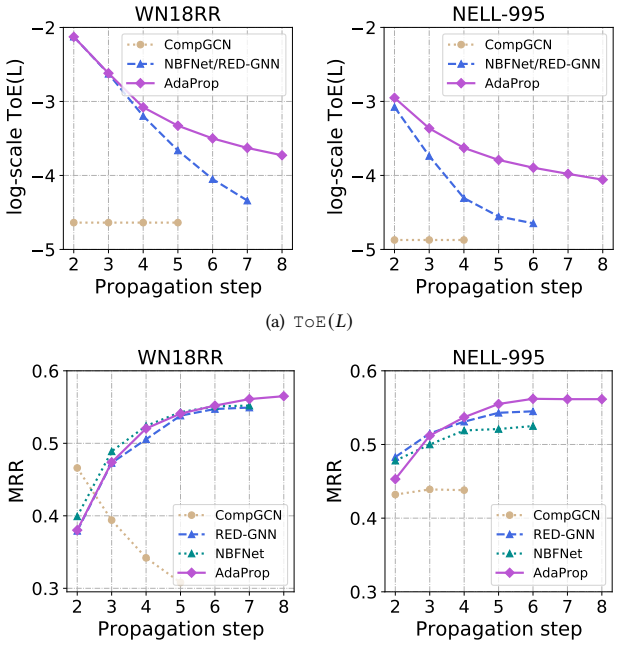
Results. The comparison of AdaProp with the transductive reasoning methods is in Table 2. First, The GNN-based methods generally perform better than the non-GNN based ones by capturing both the structural and semantic information in KGs. Second, the progressive propagating methods, i.e., NBFNet and RED-GNN, perform better than the full propagation method CompGCN, especially on larger KGs, like WN18RR, FB15k237, NELL-995 and YAGO3-10. In comparison, AdaProp achieves leading performance on WN18RR, NELL-995 and YAGO3-10 over all the baselines and slightly outperforms NBFNet on FB15k237. Besides, AdaProp also works well with competitive performances on smaller KGs Family and UMLS. To further investigate the superiority of propagation path learned by AdaProp, we analyze the properties in the following content.

Property of propagation path. To quantitatively analyze the property of propagation path, we introduce the following metric, i.e., the ratio of target over entities (ToE), to indicates the property of $\widehat{\mathcal{G}}^L_{e_q,r_q}$ with propagation steps L:

$$\texttt{ToE}(L) = \texttt{TC}(L)/\texttt{EI}(L), \tag{4}$$

where the nominator $\texttt{TC}(L) = \frac{1}{|\mathcal{Q}_{\text{tst}}|}\sum_{(e_q,r_q,e_a)\in\mathcal{Q}_{\text{tst}}}\mathbb{I}\{e_a \in \mathcal{V}^L_{e_q,r_q}\}$ measures the ratio of *target entity coverage* by the propagation path, and the denominator $\texttt{EI}(L) = \frac{1}{|\mathcal{Q}_{\text{tst}}|}\sum_{(e_q,r_q,e_a)\in\mathcal{Q}_{\text{tst}}}\left|\bigcup_{\ell=1\ldots L}\mathcal{V}^\ell_{e_q,r_q}\right|$ measures the number of *entities involved* in the propagation path. $\mathcal{Q}_{\text{tst}}$ contains queries in the testing set, $\mathcal{V}^\ell_{e_q,r_q}$ is the set of ℓ-th step entities for query $(e_q, r_q, ?)$, and $\mathbb{I}\{e_a \in \mathcal{V}^L_{e_q,r_q}\} = 1$ if e_a is in $\mathcal{V}^L_{e_q,r_q}$ otherwise 0.

We compare the value $\texttt{ToE}(L)$ and the MRR performance of different GNN-based reasoning methods with different number of propagation steps L in Figure 3. WN18RR and NELL-995 are used here. As shown, CompGCN quickly decreases on WN18RR and does not gain on NELL-995 when $L > 2$ since its $\texttt{ToE}(L)$ is fixed and very small. It will easily suffer from the problem



(a) $\texttt{ToE}(L)$

(b) Testing performance

Figure 3: Comparison of GNN-based methods w.r.t. L.

of over-smoothing [28] and quickly run out-of-memory when L increases. By contrast, RED-GNN and NBFNet have increasing performance from $L=2$ to $L=5$ by capturing the more long-range information. But they get stuck in $L>5$ when too many entities are involved. Besides, RED-GNN and NBFNet, with the same design of propagation path, have similar performance curves here. As for AdaProp, the performance consistently gets improved with larger $\texttt{ToE}(L)$ in deeper propagation steps. The benefits of a deeper propagation path can be attributed to both a larger propagation step and a deeper GNN model (we provide more details in Appendix **??**). AdaProp, with larger $\texttt{ToE}(L)$ at larger L, alleviates the problem of over-smoothing compared with other methods.

4.1.2 *Inductive setting.*

The inductive setting cares about reasoning on unseen entities. If the KG related to *Lakers* in Figure 1 is used for training, then the testing can be reason on another KG related to *Warriors* and a query *(Curry, lives_in,?)* with unseen entity *Curry*. Formally speaking, we have two KGs $\mathcal{G}_1 = (\mathcal{V}_1, \mathcal{R}, \mathcal{E}_1, \mathcal{Q}_1)$ and $\mathcal{G}_2 = (\mathcal{V}_2, \mathcal{R}, \mathcal{E}_2, \mathcal{Q}_2)$. $\mathcal{G}_1$ is used for training and validation by splitting $\mathcal{Q}_1$ into two disjoint sets $\mathcal{Q}_{\text{tra}}$ and $\mathcal{Q}_{\text{val}}$, and the unseen KG $\mathcal{G}_2$ is used for testing with $\mathcal{Q}_{\text{tst}} := \mathcal{Q}_2$. The relation sets of the two KGs are the same, while the entity sets are disjoint, i.e., $\mathcal{V}_1 \cap \mathcal{V}_2 = \emptyset$.

Datasets. Following [36, 53], we use the same subsets (4 versions each and 12 subsets in total) of WN18RR, FB15k237 and NELL-995, where each subset has a different split of training set and test set. We refer the readers to [36, 53] for the detailed splits and statistics.

Baselines. All the reasoning methods which learn entity embeddings in training (ConvE [11], QuatE [52], RotatE [34], MINERVA [9], CompGCN [38]) cannot work in this setting. Hence, we compare AdaProp with non-GNN methods that learn rules without entity embeddings, i.e., RuleN [26], NeuralLP [46] and DRUM [31].

Table 2: Transductive setting. Best performance is indicated by the bold face numbers, and the underline means the second best. "–" means unavailable results. "H@1" and "H@10" are short for Hit@1 and Hit@10 (in percentage), respectively.

type	models	Family			UMLS			WN18RR			FB15k237			NELL-995			YAGO3-10		
		MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10
non-GNN	ConvE	0.912	83.7	98.2	0.937	92.2	96.7	0.427	39.2	49.8	0.325	23.7	50.1	0.511	44.6	61.9	0.520	45.0	66.0
	QuatE	0.941	89.6	99.1	0.944	90.5	99.3	0.480	44.0	55.1	0.350	25.6	53.8	0.533	46.6	64.3	0.379	30.1	53.4
	RotatE	0.921	86.6	98.8	0.925	86.3	99.3	0.477	42.8	57.1	0.337	24.1	53.3	0.508	44.8	60.8	0.495	40.2	67.0
	MINERVA	0.885	82.5	96.1	0.825	72.8	96.8	0.448	41.3	51.3	0.293	21.7	45.6	0.513	41.3	63.7	–	–	–
	DRUM	0.934	88.1	99.6	0.813	67.4	97.6	0.486	42.5	58.6	0.343	25.5	51.6	0.532	46.0	66.2	0.531	45.3	67.6
	RNNLogic	0.881	85.7	90.7	0.842	77.2	96.5	0.483	44.6	55.8	0.344	25.2	53.0	0.416	36.3	47.8	0.554	50.9	62.2
	RLogic	–	–	–	–	–	–	0.47	44.3	53.7	0.31	20.3	50.1	–	–	–	0.36	25.2	50.4
GNNs	CompGCN	0.933	88.3	99.1	0.927	86.7	99.4	0.479	44.3	54.6	0.355	26.4	53.5	0.463	38.3	59.6	0.421	39.2	57.7
	NBFNet	0.989	**98.8**	98.9	0.948	92.0	**99.5**	0.551	49.7	66.6	0.415	32.1	**59.9**	0.525	45.1	63.9	0.550	47.9	68.6
	RED-GNN	**0.992**	**98.8**	**99.7**	0.964	94.6	99.0	0.533	48.5	62.4	0.374	28.3	55.8	0.543	47.6	65.1	0.559	48.3	68.9
	AdaProp	0.988	98.6	99.0	**0.969**	**95.6**	**99.5**	**0.562**	**49.9**	**67.1**	**0.417**	**33.1**	58.5	**0.554**	**49.3**	**65.5**	**0.573**	**51.0**	**68.5**

Table 3: Inductive setting (evaluated with Hit@10). Full evaluation results of MRR and Hit@1 are in Appendix ??.

metric	methods	WN18RR				FB15k237				NELL-995			
		V1	V2	V3	V4	V1	V2	V3	V4	V1	V2	V3	V4
Hit@10 (%)	RuleN	73.0	69.4	40.7	68.1	44.6	59.9	60.0	60.5	76.0	51.4	53.1	48.4
	Neural LP	77.2	74.9	47.6	70.6	46.8	58.6	57.1	59.3	87.1	56.4	57.6	53.9
	DRUM	77.7	74.7	47.7	70.2	47.4	59.5	57.1	59.3	87.3	54.0	57.7	53.1
	GraIL	76.0	77.6	40.9	68.7	42.9	42.4	42.4	38.9	56.5	49.6	51.8	50.6
	CoMPILE	74.7	74.3	40.6	67.0	43.9	45.7	44.9	35.8	57.5	44.6	51.5	42.1
	NBFNet	82.7	79.9	56.3	70.2	51.7	63.9	58.8	55.9	79.5	63.5	60.6	59.1
	RED-GNN	79.9	78.0	52.4	72.1	48.3	62.9	60.3	62.1	86.6	60.1	59.4	55.6
	AdaProp	**86.6**	**83.6**	**62.6**	**75.5**	**55.1**	**65.9**	**63.7**	**63.8**	**88.6**	**65.2**	**61.8**	**60.7**

For GNN-based methods, we take GraIL [36], CoMPILE [25], RED-GNN [53] and NBFNet [56] as baselines. RNNLogic [30] and RLogic [7] are not compared here since there lack source codes and reported results in this setting. We follow [53] to evaluate the ranking of target answer entity over all the negative entities, rather than 50 randomly sampled negative ones adopted in [36].

Results. As shown in Table 3, the constrained propagation methods GraIL and CoMPILE, even though have the inductive reasoning ability, are much worse than the progressive propagation methods RED-GNN and NBFNet. The patterns learned in the constrained range in GraIL and CoMPILE are not well generalized to the unseen KG. AdaProp consistently performs the best or second best in different datasets with all the split versions. This demonstrates that the learned adaptive propagation paths can be generalized to a new KG where entities are not seen during training, resulting in strong inductive reasoning ability.

4.1.3 *Running time.*

An advantage of AdaProp is that it has less computing cost due to the reduced number of entities. In this part, we compare the running time of different methods.

We show the learning curves of different GNN-based methods on transductive data WN18RR and inductive data FB15k237-v1 in Figure 4. First, the full propagation method CompGCN is the slowest method in transductive setting since it has the largest number of involved entities. Second, the constrained propagation methods, i.e., GraIL and CoMPILE, are slow in inductive setting, since their process of subgraph extraction is expensive. CoMPILE with the more complex message passing functions is even slower than GraIL. Finally, comparing AdaProp with the progressive propagation methods NBFNet and RED-GNN, the learning curve of AdaProp grows faster since it propagates with much fewer entities and the sampling cost is not high. Hence, the adaptive sampling technique is not only effective in both reasoning setting, but also efficient in propagating within fewer entities.

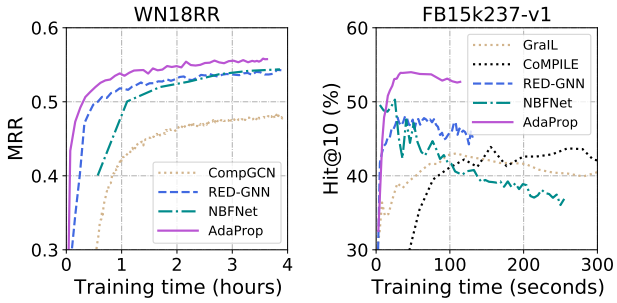


Figure 4: Learning curves of different GNN-based methods.

4.2 Understanding the sampling mechanism

In this section, we conduct a thorough ablation study via evaluating the sampling mechanism from four perspectives, i.e., the comparison on alternative sampling strategies, the importance of learnable sampling distribution, the comparison on different learning strategies, and the influence of sampled entities K. The

evaluation is based on a transductive dataset WN18RR with MRR metric and an inductive dataset FB15k237-v1 with Hit@10 metric.

4.2.1 *Sampling strategy.*

In this part, we compare the sampling strategies (with design details in Appendix B.3) discussed in Section 2.2, i.e., node-wise sampling, layer-wise sampling and subgraph sampling, and the proposed incremental sampling strategy with metrics $\texttt{EI}(L)$, $\texttt{ToE}(L)$ in Eq.(4), and testing performance. For a fair comparison, we guarantee that the different variants have the same number of propagation steps L and the similar amount of entities involved, i.e., $\texttt{EI}(L)$.

As shown in Table 4, the incremental sampling is better than the other sampling strategies in both the learned and not learned settings. The success is due to the larger $\texttt{ToE}(L)$ by preserving the previously sampled entities. The node-wise and layer-wise sampling perform much worse since the sampling there is uncontrolled. As the target entities lie close to the query entities, the subgraph sampling is relatively better than the node-wise and layer-wise sampling by propagating within the relevant local area around the query entity.

Table 4: Comparison of different sampling methods.

learn	methods	WN18RR			FB15k237-v1		
		$\texttt{EI}(L)$	$\texttt{ToE}(L)$	MRR	$\texttt{EI}(L)$	$\texttt{ToE}(L)$	Hit@10
not learned	Node-wise	4831	1.38E−4	.416	585	1.35E−3	38.9
	Layer-wise	5035	1.46E−4	.428	554	1.45E−3	37.2
	Subgraph	5098	1.57E−4	.461	578	1.50E−3	40.5
	Incremental	4954	1.61E−4	.472	559	1.52E−3	40.1
learned	Node-wise	4913	1.52E−4	.529	561	1.47E−3	50.4
	Layer-wise	4871	1.64E−4	.533	556	1.55E−3	52.4
	Incremental	4749	1.78E−4	.562	564	1.57E−3	55.1

4.2.2 *Importance of learning.*

To show the necessity of learning, we apply the same sampling distribution and learning strategy in Section 3.3 to node-wise sampling and layer-wise sampling. The subgraph sampling is hard to be learned with the proposed sampling distribution, thus not compared in this setting. By comparing the top part and bottom part in Table 4, we observe that learning is important for all the three sampling strategies. Since the entity representations can be used to measure the semantic relevance of different entities, a parameterized sampling distribution in proportion to $g(\boldsymbol{h}^{\ell}_{e_o}; \boldsymbol{\theta}^{\ell})$ helps to identify the promising targets, increasing $\texttt{ToE}(L)$ and as a result leading to better performance. Overall, the incremental sampling equipped with the learnable sampling distribution is the best choice among the sampling methods in Table 4.

4.2.3 *Learning strategies.*

In this part, we compare the variants of learning strategies. Apart from straight-through estimator (ST) introduced in Section 3.3.2, another common gradient estimators for discrete variables is the REINFORCE [27, 41], where the gradients are back-propagated by the log-trick (named as REINFORCE). Alternatively, the optimization problem in Eq.(3) can also be formed as a bi-level problem (named as Bi-level), where the sampler parameters are optimized by loss on validation data and are alternatively updated with the model parameters. The random sampler without optimizer (named as Random) is included as a reference baseline.

The evaluation results of the four learning strategies are provided in Figure 5. First, compared with the random sampler, all of the three learned distributions by Bi-level, REINFORCE and ST, are better, again verifying the importance of learning when sampling. Second, the learning curves of REINFORCE is quite unstable as this estimator has high variance. Third, the ST estimator in a single level optimization is better than the other variants with higher any-time performance.

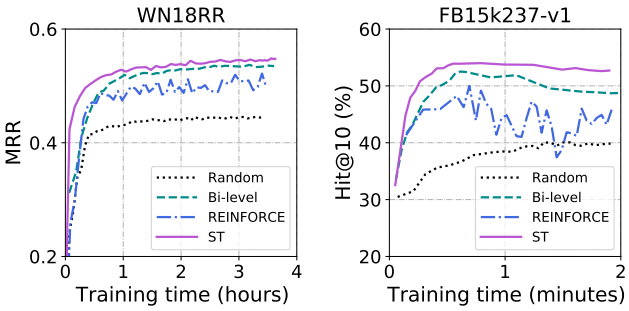


Figure 5: Comparison of learning strategies for the sampler.

4.2.4 *Influence of K.*

As discussed in Section 3.2, the number of sampled entities K bounds the number of involved entities in the propagation path. To figure out the influence of K, we show the performance of different K with respect to different propagation depth L in Figure 6. As shown, when K is very small, the results are worse than the no sampling baseline (RED-GNN) since the ratio of target entity coverage is also small. When increasing K, the performance tends to be better. However, the performance gains by sampling more entities would gradually become marginal or even worse, especially for larger L. It means that the more irrelevant information to the queries will be included with a larger K and L. Besides, a larger K will increase the training cost. Thus, K should be chosen with a moderate value.

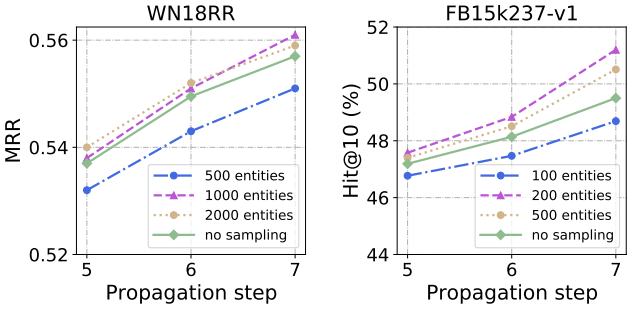
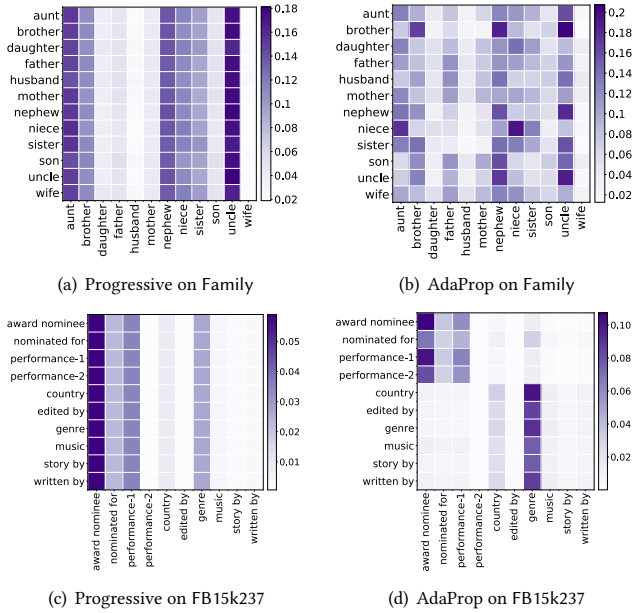


Figure 6: Ablation study with K on different L. Each line represents the performance of a given K with different L.

4.3 Case study: adaptive propagation path

In Section 3.3, we claim that the sampling distribution is semantic-aware. To demonstrate this point, we count the number of relation types in the sampled edges of the propagation path for different query relations. We plot the ratio of relation types as heatmap on two semantic friendly datasets, i.e. Family and FB15k237, in Figure 7. Specially, as there are 237 relations in FB15k237, we plot



(a) Progressive on Family (b) AdaProp on Family

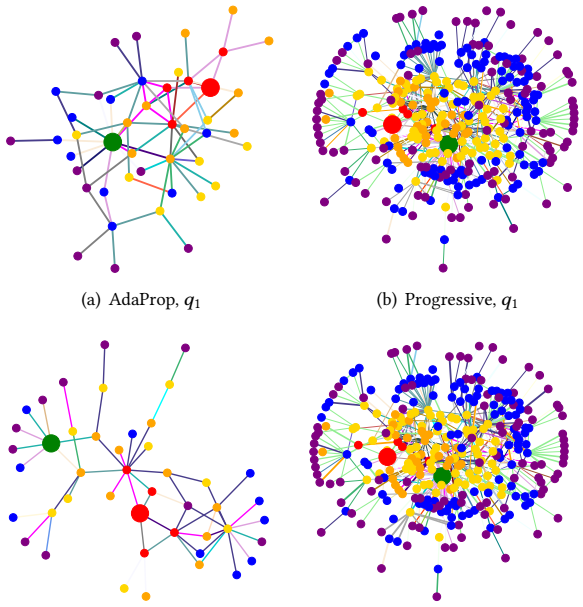
(c) Progressive on FB15k237 (d) AdaProp on FB15k237

Figure 7: Heatmaps of relation type ratios in the propagation path. Rows are different query relations and columns are relation types in the selected edges. For FB15k237**, we only show 10 selected relations that are related to "*film*" here.**

10 relations related to "*film*" in Figure 7(c) and Figure 7(d). We compare the propagation path of progressive propagation methods (NBFNet and RED-GNN have the same range) and AdaProp.

First, as claimed in the introduction, the progressive propagation methods ignore the semantic relevance between local neighborhoods and the query relation. As a result, the different rows for different query relations in Figure 7(a) and 7(c) are the same. In comparison, the heatmaps in Figure 7(b) and Figure 7(d) vary for different query relations. Second, the more frequent relation types are semantic-aware. For the heatmap in Figure 7(b), we observe that when the query relation is *brother*, columns with deeper color are all males, i.e., *brother*, *nephew* and *uncle*. When the query relation is *niece*, columns with deeper color are all females, i.e., *aunt* and *niece*. For the heatmap in Figure 7(d), the first four relations are about film award and the last six relations are about film categories. As a result, this figure shows two groups of these relations. Above evidence demonstrates that AdaProp can indeed learn semantic-aware propagation path.

In addition, we visualize the exemplar propagation paths of AdaProp and progressive propagation methods for two different queries $q_1 = (e_q, r_1, ?)$ and $q_2 = (e_q, r_2, ?)$ in Figure 8 (with more examples in in Figure **??** in Appendix), which have the same query entity e_q but different query relations ($r_1 \neq r_2$) on FB15k237-v1. As shown, there is a clear difference between the two propagation paths of q_1 and q_2 learned by AdaProp. But they are the same for the progressive propagation methods, where only the edge weights are different. Besides, the propagation paths for AdaProp have much fewer entities. The visualization cases here further demonstrate that AdaProp indeed can learn the query dependent propagation paths and is entity efficient.



(a) AdaProp, q_1 (b) Progressive, q_1

(c) AdaProp, q_2 (d) Progressive, q_2

Figure 8: Exemplar propagation paths on FB15k237-v1 **dataset. The big red and green nodes indicate the query entity and answer entity, respectively. The small nodes in red, yellow, orange, blue, purple are new entities involved in the** $1 \sim 5$ **propagation steps.**

ACKNOWLEDGMENT

Y. Zhang is supported by National Key Research and Development Plan under Grant 2021YFE0205700. Q. Yao is supported by NSF of China (No. 92270106) and Independent scientific research fund (Foshan advanced manufacturing research institute, Tsinghua University). Z. Zhou and B. Han are supported by NSFC Young Scientists Fund No. 62006202, Guangdong Basic and Applied Basic Research Foundation No. 2022A1515011652, and HKBU CSD Departmental Incentive Grant.

5 CONCLUSION

In this paper, we propose a new GNN-based KG reasoning method, called AdaProp. Different from existing GNN-based methods with hand-designed propagation path, AdaProp learns adaptive propagation path during message propagation. AdaProp contains two important components, i.e., an incremental sampling strategy where nearby targets and layer-wise connections can be preserved with linear complexity of involved entities, and a learning based sampling distribution that can identify semantically related entities during propagation and sampler is jointly optimized with the GNN models. Empirical results on several benchmarking datasets in both transductive and inductive KG reasoning settings prove the superiority of AdaProp by learning adaptive propagation path. The ablation studies show that incremental sampling is better than the other sampling strategies, and it is important as well as effective to learn the sampling distribution with straight-through gradient estimator. Case studies on the learned propagation path show that the selected entities are semantic-aware and query-dependent.

REFERENCES

[1] P. W Battaglia, J. B Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, et al. 2018. *Relational inductive biases, deep learning, and graph networks.* Technical Report. arXiv:1806.01261.

[2] Y. Bengio, N. Léonard, and A. Courville. 2013. *Estimating or propagating gradients through stochastic neurons for conditional computation.* Technical Report. arXiv:1308.3432.

[3] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. In *NeurIPS*. 2787–2795.

[4] Y. Cao, X. Wang, X. He, Z. Hu, and T. Chua. 2019. Unifying knowledge graph learning and recommendation: Towards a better understanding of user preferences. In *TheWebConf*. 151–161.

[5] J. Chen, T. Ma, and C. Xiao. 2018. FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling. In *ICLR*.

[6] X. Chen, S. Jia, and Y. Xiang. 2020. A review: Knowledge reasoning over knowledge graph. *Expert Systems with Applications* 141 (2020), 112948.

[7] K. Cheng, J. Liu, W. Wang, and Y. Sun. 2022. RLogic: Recursive Logical Rule Learning from Knowledge Graphs. In *KDD*. 179–189.

[8] W. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C. Hsieh. 2019. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *SIGKDD*. 257–266.

[9] R. Das, S. Dhuliawala, M. Zaheer, L. Vilnis, I. Durugkar, A. Krishnamurthy, A. Smola, and A. McCallum. 2017. Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning. In *ICLR*.

[10] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel. 2017. Convolutional 2D knowledge graph embeddings. In *AAAI*.

[11] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel. 2018. Convolutional 2d knowledge graph embeddings. In *AAAI*.

[12] B. Dhingra, M. Zaheer, V. Balachandran, G. Neubig, R. Salakhutdinov, and W. W Cohen. 2019. Differentiable Reasoning over a Virtual Knowledge Base. In *ICLR*.

[13] A. Feeney, R. Gupta, V. Thost, R. Angell, G. Chandu, Y. Adhikari, and T. Ma. 2021. *Relation matters in sampling: a scalable multi-relational graph neural network for drug-drug interaction prediction.* Technical Report. arXiv:2105.13975.

[14] J. Gilmer, S. S Schoenholz, P. F Riley, O. Vinyals, and G. E Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *ICML*. 1263–1272.

[15] W. Hamilton, Z. Ying, and J. Leskovec. 2017. Inductive representation learning on large graphs. In *NeurIPS*. 1024–1034.

[16] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. Emilio L. Gayo, R. Navigli, S. Neumaier, et al. 2021. Knowledge graphs. *ACM Computing Surveys (CSUR)* 54, 4 (2021), 1–37.

[17] W. Huang, T. Zhang, Y. Rong, and J. Huang. 2018. Adaptive Sampling Towards Fast Graph Representation Learning. In *NeurIPS*, Vol. 31. 4558–4567.

[18] X. Huang, J. Zhang, D. Li, and P. Li. 2019. Knowledge graph embedding based question answering. In *WSDM*. 105–113.

[19] E. Jang, S. Gu, and B. Poole. 2017. Categorical Reparameterization with Gumbel-Softmax. In *ICLR*.

[20] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S Yu. 2020. *A survey on knowledge graphs: Representation, acquisition and applications.* Technical Report. arXiv:2002.00388.

[21] D. P Kingma and J. Ba. 2014. *Adam: A method for stochastic optimization.* Technical Report. arXiv:1412.6980.

[22] T. N Kipf and M. Welling. 2016. Semi-supervised classification with graph convolutional networks. In *ICLR*.

[23] S. Kok and P. Domingos. 2007. Statistical predicate invention. In *ICML*. 433–440.

[24] W. Kool, H. Van Hoof, and M. Welling. 2019. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement. In *ICML*. PMLR, 3499–3508.

[25] S. Mai, S. Zheng, Y. Yang, and H. Hu. 2021. Communicative Message Passing for Inductive Relation Reasoning.. In *AAAI*. 4294–4302.

[26] C. Meilicke, M. Fink, Y. Wang, D. Ruffinelli, R. Gemulla, and H. Stuckenschmidt. 2018. Fine-grained evaluation of rule-and embedding-based systems for knowledge graph completion. In *ISWC*. Springer, 3–20.

[27] A. Mnih and K. Gregor. 2014. Neural variational inference and learning in belief networks. In *ICML*. PMLR, 1791–1799.

[28] K. Oono and T. Suzuki. 2019. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. In *ICLR*.

[29] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. 2017. Automatic differentiation in PyTorch. In *ICLR*.

[30] M. Qu, J. Chen, L. Xhonneux, Y. Bengio, and J. Tang. 2021. RNNLogic: Learning Logic Rules for Reasoning on Knowledge Graphs. In *ICLR*.

[31] A. Sadeghian, M. Armandpour, P. Ding, and D. Z. Wang. 2019. DRUM: End-To-End Differentiable Rule Mining On Knowledge Graphs. In *NeurIPS*. 15347–15357.

[32] M. Schlichtkrull, T. N Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling. 2018. Modeling relational data with graph convolutional networks. In *ESWC*. Springer, 593–607.

[33] F. Suchanek, G. Kasneci, and G. Weikum. 2007. Yago: A core of semantic knowledge. In *The WebConf*. 697–706.

[34] Z. Sun, Z. Deng, J. Nie, and J. Tang. 2019. Rotate: Knowledge graph embedding by relational rotation in complex space. In *ICLR*.

[35] R. S Sutton and A. Barto. 2018. *Reinforcement learning: An introduction.* MIT press.

[36] K. K Teru, E. Denis, and W. L Hamilton. 2020. *Inductive Relation Prediction by Subgraph Reasoning.* Technical Report. arXiv:1911.06962.

[37] K. Toutanova and D. Chen. 2015. Observed versus latent features for knowledge base and text inference. In *PWCVSMC*. 57–66.

[38] S. Vashishth, S. Sanyal, V. Nitin, and P. Talukdar. 2019. Composition-based multi-relational graph convolutional networks.

[39] Q. Wang, Z. Mao, B. Wang, and L. Guo. 2017. Knowledge graph embedding: A survey of approaches and applications. *TKDE* 29, 12 (2017), 2724–2743.

[40] X. Wang, X. He, Y. Cao, M. Liu, and T. Chua. 2019. Kgat: Knowledge graph attention network for recommendation. In *SIGKDD*. 950–958.

[41] R. J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning Journal* 8, 3-4 (1992), 229–256.

[42] C. Wilt, J. Thayer, and W. Ruml. 2010. A comparison of greedy search algorithms. In *Proceedings of the International Symposium on Combinatorial Search*, Vol. 1. 129–136.

[43] S. Xie and S. Ermon. 2019. Reparameterizable Subset Sampling via Continuous Relaxations. In *IJCAI*.

[44] W. Xiong, T. Hoang, and W. Wang. 2017. DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning. In *EMNLP*. 564–573.

[45] X. Xu, W. Feng, Y. Jiang, X. Xie, Z. Sun, and Z. Deng. 2019. Dynamically Pruned Message Passing Networks for Large-Scale Knowledge Graph Reasoning. In *ICLR*.

[46] F. Yang, Z. Yang, and W. W Cohen. 2017. Differentiable learning of logical rules for knowledge base reasoning. In *NeurIPS*. 2319–2328.

[47] Z. Ye, Y. Jaya Kumar, G. Sing, F. Song, and J. Wang. 2022. A comprehensive survey of graph neural networks for knowledge graphs. *IEEE Access* 10 (2022), 75729–75741.

[48] M. Yoon, T. Gervet, B. Shi, S. Niu, Q. He, and J. Yang. 2021. Performance-Adaptive Sampling Strategy Towards Fast and Accurate Graph Neural Networks. In *SIGKDD*. 2046–2056.

[49] Y. Yu, K. Huang, C. Zhang, L. M Glass, J. Sun, and C. Xiao. 2021. SumGNN: multi-typed drug interaction prediction via efficient knowledge graph summarization. *Bioinformatics* 37, 18 (2021), 2988–2995.

[50] H. Zeng, M. Zhang, Y. Xia, A. Srivastava, A. Malevich, R. Kannan, V. Prasanna, L. Jin, and R. Chen. 2021. Decoupling the Depth and Scope of Graph Neural Networks. In *NeurIPS*, Vol. 34.

[51] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna. 2019. GraphSAINT: Graph Sampling Based Inductive Learning Method. In *ICLR*.

[52] S. Zhang, Y. Tay, L. Yao, and Q. Liu. 2019. Quaternion knowledge graph embeddings. *NeurIPS* 32 (2019).

[53] Y. Zhang and Q. Yao. 2022. Knowledge Graph Reasoning with Relational Directed Graph. In *TheWebConf*.

[54] Y. Zhang, Q. Yao, and L. Chen. 2020. Interstellar: Searching Recurrent Architecture for Knowledge Graph Embedding. In *NeurIPS*, Vol. 33.

[55] Y. Zhang, Q. Yao, W. Dai, and L. Chen. 2020. AutoSF: Searching scoring functions for knowledge graph embedding. In *ICDE*. IEEE, 433–444.

[56] Z. Zhu, Z. Zhang, L. Xhonneux, and J. Tang. 2021. Neural Bellman-Ford Networks: A General Graph Neural Network Framework for Link Prediction. In *NeurIPS*.

[57] D. Zou, Z. Hu, Y. Wang, S. Jiang, Y. Sun, and Q. Gu. 2019. Layer-Dependent Importance Sampling for Training Deep and Large Graph Convolutional Networks. *NeurIPS* 32 (2019), 11249–11259.

Notations. In this paper, vectors are denoted by lowercase boldface, e.g., $\boldsymbol{m}, \boldsymbol{h}, \boldsymbol{w}$. Matricies are denoted by uppercase boldface, e.g., $\boldsymbol{W}$. Sets are denoted by script fonts, e.g,. $\mathcal{E}, \mathcal{V}$. As introduced in Section 3, a knowledge graph is in the form of $\mathcal{K} = \{\mathcal{V}, \mathcal{R}, \mathcal{E}, \mathcal{Q}\}$.

A DETAILS OF WORKS IN THE LITERATURE

A.1 Summary of MESS$(\cdot)$ and AGG$(\cdot)$

The two functions of the GNN-based methods are summarized in Table 5. The main differences of these methods are the combinators for entity and relation representations on the edges, the operators on the different representations, and the attention weights. Recent works [38, 56] have shown that the different combinators and operators only have slight influence on the performance. Comparing GraIL and RED-GNN, even though the message functions and aggregation functions are similar, their empirical performances are quite different with different propagation patterns. Hence, the design of propagation path is the *key factor* influencing the performance of different methods.

A.2 Discussion on constrained propagation

In KG reasoning, the key objective is to find the target answer entity e_a given $(e_q, r_q, ?)$. To fulfill this goal, the model is required to evaluate multiple target answer entities. Since all the entities in the KG can be potential answers, we need to evaluate $|\mathcal{V}|$ entities for a single query $(e_q, r_q, ?)$. However, for the constrained propagation method GraIL [36] and CoMPILE [25], the propagation path depends on both the query entity e_q and the answer entity e_a, i.e., $\mathcal{V}^\ell = \mathcal{V}^L_{e_q,e_a}$. Hence, it requires $|\mathcal{V}|$ times of forward passes to evaluate all the entities for a single query, which is extremely expensive when the number of entities is large. For the settings of transductive reasoning, all datasets have tens of thousands of entities. Thus, running GraIL or CoMPILE on them is intractable.

B DETAILS OF THE SAMPLING METHOD

B.1 Augmentation with inverse relations

As aforementioned, predicting missing head in KG can also be formulated in this way by adding inverse relations. This is achieved by augmenting the original KG by the reverse relations and the identity relation, as adopted in path-based methods [30, 31]. Taking Figure 1 as example, we add the reverse relation "part_of inv" to the original relation "part_of", which leads to a new triplet *(Lakers, part_of inv, LeBron)*. We also add the identity relation as the self-loop, which leads to new triplets, such as *(LeBron, identity, LeBron)*.

B.2 Proofs of propositions

B.2.1 Proposition 1. Since $|\mathcal{V}^0_{e_q,r_q}| = 1$, $\mathcal{V}^\ell_{e_q,r_q} := \mathcal{V}^{\ell-1}_{e_q,r_q} \cup \texttt{SAMP}(\overline{\mathcal{V}}^\ell_{e_q,r_q})$ and $|\texttt{SAMP}(\overline{\mathcal{V}}^\ell_{e_q,r_q})| \le K$, we have

$$\mathcal{V}^\ell_{e_q,r_q} \le 1 + \ell \cdot K,$$

for $\ell = 1 \dots L$. Meanwhile, with the constraint $\mathcal{V}^0_{e_q,r_q} \subseteq \mathcal{V}^1_{e_q,r_q} \cdots \subseteq \mathcal{V}^L_{e_q,r_q}$, we have

$$\Big|\bigcup_{\ell=0\dots L} \mathcal{V}^\ell_{e_q,r_q}\Big| = \big|\mathcal{V}^L_{e_q,r_q}\big| \le 1 + L \cdot K.$$

Namely, the number of involved entities is bounded by $O(L \cdot K)$.

B.2.2 Proposition 2. Since $\mathcal{V}^\ell_{e_q,r_q} := \mathcal{V}^{\ell-1}_{e_q,r_q} \cup \texttt{SAMP}(\overline{\mathcal{V}}^\ell_{e_q,r_q})$, we have $\mathcal{V}^{\ell-1}_{e_q,r_q} \subseteq \mathcal{V}^\ell_{e_q,r_q}$. With the introduction of identity relation in Section B.1, for each $e \in \mathcal{V}^{\ell-1}_{e_q,r_q}$, there exist at least one entity $e \in \mathcal{V}^\ell_{e_q,r_q}$ such that $(e, \textit{identity}, e) \in \mathcal{E}^\ell$. Thus, the connection can be preserved.

B.3 Adapting sampling methods for KG

When modeling the homogeneous graphs, the sampling methods are designed to solve the scalability problem. Besides, they are mainly used in the node classification task, which is quite different from the link-level KG reasoning task here. Hence, we introduce the adaptations we made to use the sampling methods designed for homogeneous graphs in the KG reasoning scenarios as follows.

- Node-wise sampling. The design in this type is based on GraphSAGE [15]. Given $\mathcal{V}^{\ell-1}_{e_q,r_q}$, we sample K neighboring entities from $\mathcal{N}(e)$ for entities $e \in \mathcal{V}^{\ell-1}_{e_q,r_q}$. Then the union of these sampled entities forms $\mathcal{V}^\ell_{e_q,r_q}$ for next step propagation. For the not-learned version, we randomly sample from $\mathcal{N}(e)$. For the learned version, we sample from $\mathcal{N}(e)$ based on the distribution $p^\ell(e) \sim g(\boldsymbol{h}^\ell_e; \boldsymbol{\theta}^\ell)$.
- Layer-wise sampling. The design in this type is based on LADIES [57]. Given $\mathcal{V}^{\ell-1}_{e_q,r_q}$, we firstly collect the set of neighboring entities $\mathcal{N}(\mathcal{V}^{\ell-1}) = \bigcup_{e\in\mathcal{V}^{\ell-1}_{e_q,r_q}} \mathcal{N}(e)$. Then, we sample K entities without replacement from $\mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q})$ as $\mathcal{V}^\ell$ for next step propagation. For the not-learned version, we sample according to the degree distribution for entities in $\mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q})$. For the learned version, we sample from $\mathcal{N}(\mathcal{V}^{\ell-1}_{e_q,r_q})$ based on the distribution $p^\ell(e) \sim g(\boldsymbol{h}^\ell_e; \boldsymbol{\theta}^\ell)$.
- Subgraph sampling. Given the query $(e_q, r_q, ?)$, we generate multiple random walks starting from the query entity e_q and use the sampled entities to induce the subgraph $\mathcal{V}_{e_q}$. Then, the propagation starts from e_q like the progressive propagation, but it is constrained within the range of $\mathcal{V}_{e_q}$. Since learning to sample the subgraph is challenging, we only evaluate the subgraph sampling in the not-learned version.

B.4 Implementation of ST estimator

The key idea of straight-through (ST) estimator is to back-propagate through the sampling signal as if it was the identity function. Given the sampled entities $e_o \in \widetilde{\mathcal{V}}^\ell_{e_q,r_q}$ as well as their sampling probability $p^\ell(e)$, the derivative of the sampling parameters can be obtained by multiplying $p^\ell(e)$ with the hidden states $\boldsymbol{h}^\ell_e$. Specifically, rather than directly use the $\boldsymbol{h}^\ell_e$ to compute messages, we use

$$\boldsymbol{h}^\ell_e := \big(1 - \texttt{no\_grad}(p^\ell(e)) + p^\ell(e)\big) \cdot \boldsymbol{h}^\ell_e, \tag{5}$$

where $\texttt{no\_grad}(p^\ell(e))$ means that the back-propagation signals will not go through this term. In this approach, the forward computation will not be influenced, while the backward signals can go through the multiplier $p^\ell(e)$. Besides, the backward signals, even though not exactly equivalent to the true gradient, is in positive propagation to it. Hence, we can use such a backward signal to approximately estimate the gradient of $\mathcal{L}$ with regard to $\boldsymbol{\theta}$.

Table 5: Summary of the GNN functions for message propagation. The values in {} represent different operation choices that are tuned as hyper-parameters for different datasets.

method	$\boldsymbol{m}^{\ell}_{(e_s,r,e_o)} := \text{MESS}(\cdot)$	$\boldsymbol{h}^{\ell}_{e_o} := \text{AGG}(\cdot)$
R-GCN [32]	$\boldsymbol{W}^{\ell}_{r}\boldsymbol{h}^{\ell-1}_{e_s}$, where $\boldsymbol{W}_r$ depends on the relation r	$\boldsymbol{W}^{\ell}_{o}\boldsymbol{h}^{\ell-1}_{e_o}+\sum_{e_o\in\mathcal{N}(e_s)}\frac{1}{c_{o,r}}\boldsymbol{m}^{\ell}_{(e_s,r,e_o)}$
CompGCN [38]	$\boldsymbol{W}^{\ell}_{\lambda(r)}\{-,*,\star\}(\boldsymbol{h}^{\ell-1}_{e_s},\boldsymbol{h}^{\ell}_{r})$, where $\boldsymbol{W}^{\ell}_{\lambda(r)}$ depends on the direction of r	$\sum_{e_o\in\mathcal{N}(e_s)}\boldsymbol{m}^{\ell}_{(e_s,r,e_o)}$
GraIL [36]	$\alpha^{\ell}_{(e_s,r,e_o)\|r_q}(\boldsymbol{W}^{\ell}_{1}\boldsymbol{h}^{\ell-1}_{e_s}+\boldsymbol{W}^{\ell}_{2}\boldsymbol{h}^{\ell-1}_{e_o})$, where $\alpha^{\ell}_{(e_s,r,e_o)\|r_q}$ is the attention weight.	$\boldsymbol{W}^{\ell}_{o}\boldsymbol{h}^{\ell-1}_{e_o}+\sum_{e_o\in\mathcal{N}(e_s)}\frac{1}{c_{o,r}}\boldsymbol{m}^{\ell}_{(e_s,r,e_o)}$
NBFNet [56]	$\boldsymbol{W}^{\ell}\{+,*,\circ\}(\boldsymbol{h}^{\ell-1}_{e_s},\boldsymbol{w}_q(r,r_q))$, where $\boldsymbol{w}_q(r,r_q)$ is a query-dependent weight vector	$\{\texttt{Sum},\texttt{Mean},\texttt{Max},\texttt{PNA}\}_{e_o\in\mathcal{N}(e_s)}\boldsymbol{m}^{\ell}_{(e_s,r,e_o)}$
RED-GNN [53]	$\alpha^{\ell}_{(e_s,r,e_o)\|r_q}(\boldsymbol{h}^{\ell-1}_{e_s}+\boldsymbol{h}^{\ell}_{r})$, where $\alpha^{\ell}_{(e_s,r,e_o)\|r_q}$ is the attention weight	$\sum_{e_o\in\mathcal{N}(e_s)}\boldsymbol{m}^{\ell}_{(e_s,r,e_o)}$
AdaProp	$\alpha^{\ell}_{(e_s,r,e_o)\|r_q}\{+,*,\circ\}(\boldsymbol{h}^{\ell-1}_{e_s},\boldsymbol{h}^{\ell}_{r})$, where $\alpha^{\ell}_{(e_s,r,e_o)\|r_q}$ is as in [53].	$\{\texttt{Sum},\texttt{Mean},\texttt{Max}\}_{e_o\in\mathcal{N}(e_s)}\boldsymbol{m}^{\ell}_{(e_s,r,e_o)}$

Table 7: Optimal hyper-parameter (HP) for each dataset. "BS" is short for batch size.

HP	Family	UMLS	WN18RR	FB15k237	NELL-995	YAGO3-10
L	8	5	8	7	6	8
K	100	100	1000	2000	2000	1000
τ	1.0	1.0	0.5	2.0	0.5	1.0
BS	20	10	100	50	20	5

	WN18RR				FB15k237				NELL-995			
HP	V1	V2	V3	V4	V1	V2	V3	V4	V1	V2	V3	V4
L	7	7	7	6	8	6	7	7	6	5	8	5
K	200	100	200	200	200	200	200	500	200	200	500	100
τ	0.5	0.5	1.0	1.0	0.5	2.0	1.0	1.0	0.5	1.0	1.0	2.0
BS	20	20	50	50	50	20	20	50	20	50	20	50

C FURTHER MATERIALS OF EXPERIMENTS

C.1 Hop distributions of datasets

We summarize the distance distribution of test queries in Table 6 for the largest four KGs. Here, the distance for a query triple (e_q, r_q, e_a) is calculated as the length of the shortest path connecting e_q and e_a from the training triples. As shown, *most answer entities are close to query entities*, i.e., within the 3-hop neighbors of query entities.

Table 6: Distance distribution (in %) of queries in Q_{tst}.

distance	1	2	3	4	5	>5
WN18RR	34.9	9.3	21.5	7.5	8.9	17.9
FB15k237	0.0	73.4	25.8	0.2	0.1	0.5
NELL-995	40.9	17.2	36.5	2.5	1.3	1.6
YAGO3-10	56.0	12.9	30.1	0.5	0.1	0.4

C.2 Hyper-parameters

We select the optimal hyper-parameters for evaluation based on the MRR metric or the Hit@10 metric on validation set for transductive setting or indcutive setting, respectively. We provide the hyper-parameters of L, K, τ and batch size in Table 7. We observe that (1) In most cases, we have a larger number of propagation steps with $L \geq 6$. (2) In the transductive setting, the values of K are larger since the KGs are larger. But in the inductive setting, K's are generally small. This means that K is not as large as possible. More irrelevant entities will influence the performance. (3) There is no much regularity in the choices of τ, which depends on specific KGs.

Other materials

Due to space limitation, the other parts of the Appendix are in the full version of this paper[2].

[2]https://arxiv.org/pdf/2205.15319.pdf